

Listas de Exercícios — Projeto de Algoritmos

Funções, Recursividade, Força Bruta, Divisão e Conquista, Algoritmos Gulosos e Programação Dinâmica

Prof. Dr. Guilherme Apolinário Silva Novaes
UNISANTOS

Instruções gerais, leia tudo antes de começar

1. **Grupo sim, entrega individual.** Discuta e resolva junto com seu grupo durante a aula, isso é esperado e faz parte do aprendizado. Mas cada um entrega **o seu próprio material**, escrito por si. Se duas entregas idênticas aparecerem, **os dois** terão que explicar o código na minha frente.
2. **O que fazer em aula:** os exercícios **[Fácil]** (3 por tópico) são para serem feitos **em aula, em grupo**. Os exercícios **[Médio]** e **[Difícil]** são **desafios para estudar em casa**, ninguém é obrigado a terminar em aula, mas tentar depois é o que faz a diferença na prova.
3. **Entrega no final de cada aula:**

Aula	O que entregar	Conteúdo
Aula 1 (revisão 1)	Lista A	3 [Fácil] de cada um dos 6 tópicos
Aula 2 (revisão 2)	Lista B	3 [Fácil] de cada um dos 6 tópicos
Aula 3 (revisão 3)	Lista C	3 [Fácil] de cada um dos 6 tópicos

A entrega é **separada por lista**: na entrega da Lista A só aceito Lista A; na da Lista B, só Lista B; e assim por diante. **Confira o rótulo de cada exercício** (A-F1, A-R3, B-D5, C-P4, ...) antes de colocar na folha, entrega errada não será trocada depois.

Formato da entrega: um único arquivo .zip contendo **todos os arquivos .c daquela lista**. Cada código-fonte deve estar em um arquivo separado, nomeado com o rótulo do exercício (ex.: A-F1.c, A-R3.c, B-D5.c). O arquivo zip deve se chamar Lista<Letra>_<SeuNome>.zip (ex.: ListaA_JoaoSilva.zip). **Zip sem os arquivos .c, com executável no lugar do código, ou fora do padrão de nomes não será aceito.**

4. **Faltou entregar? Acumula.** Se você não terminou a Lista A na Aula 1, ela *acumula*: entregue na Aula 2 ou na Aula 3, mas **sempre identificada como Lista A e separada da lista da vez** (um zip para cada lista, na dúvida). Não misture exercícios de listas diferentes na mesma entrega.
5. **Regra do zero:** entregar **nenhum** exercício de uma lista = nota **zero** naquela lista. Entrega parcial vale! Não precisa completar tudo, faça o que conseguir e entregue.

6. Ficou na dúvida? (a) Primeiro pergunte ao seu grupo; (b) se o grupo inteiro estiver travado, pergunte a outro grupo; (c) se **a sala toda** tiver a mesma dúvida, **pule o exercício**, anote o número dele (ex.: “B-D4 em dúvida”) na folha e siga em frente, resolvemos juntos quando eu voltar. **Não fique mais do que 5 minutos parado no mesmo exercício.**

7. Antes de entregar, confira:

O **.zip** contém **todos** os arquivos **.c** da lista, um por exercício feito, nomeados pelo rótulo (A-F1.c, A-F2.c, ...);

Cada programa **compila** e a saída bate **exatamente** com os **Casos de teste** do enunciado;

Nome completo e identificação estão corretos no nome do **.zip** (Lista<Letra>_<SeuNome>.zip);

Você consegue explicar com suas palavras o que cada trecho do código faz, posso chamar qualquer pessoa para explicar a entrega.

8. Os Casos de teste são a especificação. Não é “parecido” que vale: a saída do seu programa tem que ser **igualzinha** à do enunciado, caractere por caractere, inclusive nos casos de borda (**n** = 0, valor não encontrado, entrada vazia, número negativo). Teste **todos** os casos antes de entregar.

9. Material de consulta: podem usar os slides da disciplina, o próprio caderno e códigos que vocês mesmos escreveram. (*Professor: edite aqui se quiser liberar ou proibir livros/internet.*)

10. Linguagem: C, exatamente como nos slides (protótipos, ponteiros, **struct**, etc.). Código legível, indentado e com nomes de variáveis claros conta pontos, código “torto” que funciona recebe desconto.

11. Não sabe por onde começar? Releia o enunciado procurando por: (a) a **assinatura da função** (nome, tipos, parâmetros); (b) o que a **main** deve fazer; (c) os **Casos de teste**. Quase sempre dá para montar o esqueleto do programa (includes + protótipo + main vazia) só copiando essas informações, e aí a lógica fica mais fácil de enxergar.

1 Lista A

1. Funções

1. **[Fácil] Linha de separação.** Escreva uma função `void linha(void)` que imprime uma linha com exatamente 80 caracteres '-'. Escreva o protótipo da função antes da `main`. A `main` não lê nada do teclado e produz exatamente 3 linhas de saída, chamando `linha()` duas vezes.

Saída: linha com 80 hífens; a frase `Um programa em C` centralizada com tabulações (`\t`); outra linha com 80 hífens.

Casos de teste:

```
-----  
Um programa em C  
-----
```

2. **[Fácil] Soma de dois números.** Escreva a função `int soma(int a, int b)` que **devolve** (com `return`) a soma dos dois parâmetros. A `main` lê dois inteiros e imprime `Soma: X`, onde `X` é o valor devolvido por `soma(n1, n2)`.

Entrada: dois inteiros separados por espaço.

Saída: a linha `Soma: X`.

Casos de teste:

```
Entrada: 7 3      -> Saida: Soma: 10  
Entrada: -4 10    -> Saida: Soma: 6  
Entrada: 0 0      -> Saida: Soma: 0
```

3. **[Fácil] Dobro (passagem por valor).** Escreva a função `void dobro(int x)` que imprime `Dobro: Y`, onde `Y = 2*x`. A `main` lê um inteiro `n`, chama `dobro(n)` e depois imprime `Numero informado: n` (o valor original de `n`).

Entrada: um inteiro `n`.

Saída: duas linhas: `Dobro: 2n` e `Numero informado: n`.

Casos de teste:

```
Entrada: 6      -> Saida: Dobro: 12 / Numero informado: 6  
Entrada: -3     -> Saida: Dobro: -6 / Numero informado: -3
```

Perguntas: o valor de `n` na `main` foi alterado pela chamada? Justifique com o conceito de passagem por valor (cópia temporária).

4. **[Médio] Troca de valores por referência.** Escreva a função `void troca(int *pa, int *pb)` que troca os conteúdos das variáveis apontadas pelos dois ponteiros, usando uma variável auxiliar. A `main` declara `a = 10` e `b = 20` fixos no código.

Entrada: nenhuma (valores fixos na `main`).

Saída: duas linhas: valores de `a` e `b` **antes** e **depois** de `troca(&a, &b)`.

Casos de teste:

Saida: a = 10, b = 20
a = 20, b = 10

Perguntas: por que é preciso passar os **endereços** (&a, &b)? O que aconteceria se a e b fossem passados por valor?

5. **[Difícil] Estatísticas de um vetor com argc/argv.** Escreva a função void `estatisticas(float vet[], int n, float *media, float *maior)` que calcula a média e o maior valor do vetor, devolvendo os dois resultados **por referência**. A main recebe n e os n elementos como argumentos da linha de comando (`argc/argv`), convertendo cada argumento com `atof`.

Entrada: linha de comando: `./prog 5 4.5 7.0 1.5 9.0 3.5` (o 1º argumento após o nome do programa é n; seguem-se os n elementos).

Saída: duas linhas: Media: X (1 casa decimal) e Maior: Y.

Casos de teste:

```
./prog 5 4.5 7.0 1.5 9.0 3.5 -> Media: 5.1 / Maior: 9.0
./prog 3 1.0 2.0 4.0          -> Media: 2.3 / Maior: 4.0
```

2. Recursividade

1. **[Fácil] Fatorial.** Escreva a função recursiva `long long fat(int n)`: solução trivial `fat(0) = 1`; solução geral `fat(n) = n * fat(n-1)`. A main lê n e imprime Fatorial: X.

Entrada: um inteiro n >= 0.

Saída: a linha Fatorial: X.

Casos de teste:

```
Entrada: 0    -> Saida: Fatorial: 1
Entrada: 1    -> Saida: Fatorial: 1
Entrada: 5    -> Saida: Fatorial: 120
Entrada: 10   -> Saida: Fatorial: 3628800
```

Perguntas: identifique no código a solução trivial e a solução geral.

2. **[Fácil] Potenciação.** Escreva a função recursiva `int pot(int x, int y)` que calcula x^y para inteiros não negativos: solução trivial $x^0 = 1$; solução geral $x^y = x \cdot x^{y-1}$. Trate y = 0 antes de recorrer.

Entrada: dois inteiros x e y (y >= 0).

Saída: a linha Resultado: X.

Casos de teste:

```
Entrada: 2 0    -> Saida: Resultado: 1
Entrada: 2 10   -> Saida: Resultado: 1024
Entrada: 3 4    -> Saida: Resultado: 81
Entrada: 5 3    -> Saida: Resultado: 125
```

3. **[Fácil] Comprimento de string.** Escreva a função recursiva `int compr(const char *str)` que devolve o número de caracteres da string. Solução trivial: `*str == '\0'` devolve 0; solução geral: `1 + compr(str + 1)`. **Proibido** usar `strlen`.

Entrada: uma string (sem espaços).

Saída: a linha Comprimento: X.

Casos de teste:

Entrada: unisantos -> Saida: Comprimento: 9

Entrada: a -> Saida: Comprimento: 1

Entrada: (string vazia) -> Saida: Comprimento: 0

4. **[Médio] Soma dos n primeiros naturais.** Escreva a função recursiva `int soma_nat(int n)` que devolve $1 + 2 + \dots + n$: solução trivial `soma_nat(0) = 0`; solução geral `soma_nat(n) = n + soma_nat(n-1)`.

Entrada: um inteiro $n \geq 0$.

Saída: a linha Soma: X.

Casos de teste:

Entrada: 0 -> Saida: Soma: 0

Entrada: 1 -> Saida: Soma: 1

Entrada: 5 -> Saida: Soma: 15

Entrada: 100 -> Saida: Soma: 5050

5. **[Difícil] Torres de Hanói.** Escreva a função recursiva `void hanoi(int n, char origem, char destino, char auxiliar)` que imprime cada movimento no formato `<origem> -> <destino>`. Lógica: (1) mova $n-1$ discos de `origem` para `auxiliar`; (2) imprima o movimento do disco restante `origem -> destino`; (3) mova $n-1$ discos de `auxiliar` para `destino`. A main chama `hanoi(n, 'A', 'C', 'B')` lendo n .

Entrada: um inteiro $n \geq 1$.

Saída: uma linha por movimento, no formato `X -> Y`.

Casos de teste:

Entrada: 1 -> Saida: A -> C

Entrada: 2 -> Saida: A -> B / A -> C / B -> C

Entrada: 3 -> Saida: A -> C / A -> B / C -> B / A -> C /

B -> A / B -> C / A -> C

Perguntas: (a) quantos movimentos para $n = 1, 2, 3$? (b) qual a fórmula do número mínimo de movimentos em função de n ? (c) desenhe a pilha de execução para `hanoi(2, 'A', 'C', 'B')`.

3. Força Bruta

1. **[Fácil]** Busca sequencial. Escreva `int busca_sequencial(int vet[], int n, int x)` que percorre o vetor do início ao fim e retorna o índice da **primeira** ocorrência de `x`, ou `-1` se não existir. A `main` lê `n`, os `n` elementos e `x` (nessa ordem).

Entrada: `n`; os `n` inteiros; o valor `x`.

Saída: o índice encontrado ou `-1`.

Casos de teste:

```
Entrada: 6 / 3 7 1 9 7 4 / 9    -> Saída: 3
Entrada: 6 / 3 7 1 9 7 4 / 7    -> Saída: 1    (primeira ocorrência!)
Entrada: 6 / 3 7 1 9 7 4 / 5    -> Saída: -1
Entrada: 1 / 42 / 42            -> Saída: 0
```

2. **[Fácil]** Contagem de ocorrências. Escreva `int conta(int vet[], int n, int x)` que conta quantas vezes `x` aparece no vetor. A `main` lê os dados e imprime `Ocorrencias: X`.

Entrada: `n`; os `n` elementos; `x`.

Saída: a linha `Ocorrencias: X`.

Casos de teste:

```
Entrada: 6 / 3 7 1 9 7 4 / 7    -> Saída: Ocorrencias: 2
Entrada: 6 / 3 7 1 9 7 4 / 5    -> Saída: Ocorrencias: 0
Entrada: 5 / 2 2 2 2 2 / 2      -> Saída: Ocorrencias: 5
```

3. **[Fácil]** Ordenação por seleção. Escreva `void selecao_direta(int n, int a[])`: para cada `i` de 0 até `n-2`, encontre o índice `k` do menor elemento entre `i` e `n-1` e troque `a[i]` com `a[k]`. A `main` imprime o vetor após cada passo (troca realizada).

Entrada: `n`; os `n` elementos.

Saída: `n-1` linhas com o vetor após cada passo.

Casos de teste:

```
Entrada: 4 / 3 1 4 2
Saída:   1 3 4 2    (passo 0: menor=1, trocado com a[0]=3)
         1 2 4 3    (passo 1)
         1 2 3 4    (passo 2)
```

4. **[Médio]** Busca de palavra em texto (string matching).

Escreva `bool string_match(const char *s1, const char *s2)` ($m \leq n$) que verifica se `s2` é substring de `s1` por força bruta: para cada posição `i` de 0 até `n - m`, compare `s2[j]` com `s1[i + j]`; se os `m` caracteres coincidirem, devolva `true`; ao final, `false`. A `main` imprime `Encontrada` ou `Nao encontrada`.

Entrada: duas strings `s1` e `s2` (sem espaços).

Saída: `Encontrada` ou `Nao encontrada`.

Casos de teste:

Entrada: unisantos san -> Saída: Encontrada
 Entrada: unisantos tnt -> Saída: Nao encontrada
 Entrada: unisantos uni -> Saída: Encontrada
 Entrada: aaa aa -> Saída: Encontrada
 Entrada: abc abcd -> Saída: Nao encontrada (m > n)

5. **[Difícil]** Mochila booleana (recursiva). Escreva `int mochila_rec(int p[], int v[], int c, int n, int item)` que devolve o maior valor que cabe na mochila de capacidade `c`. Regra: se `item >= n` ou `c == 0`, devolva 0; se `p[item] > c`, **obrigatoriamente** pule o item (`mochila_rec(p, v, c, n, item+1)`); senão devolva o máximo entre `v[item] + mochila_rec(p, v, c - p[item], n, item+1)` (coloca) e `mochila_rec(p, v, c, n, item+1)` (não coloca). A primeira chamada é `mochila_rec(p, v, c, n, 0)`.

Entrada: capacidade `c`; `n`; os `n` pesos; os `n` valores (nesta ordem).

Saída: a linha Valor maximo: X.

Casos de teste:

Entrada: 50 / 5 / 40 30 20 10 20 / 840 600 400 100 300
 Saída: Valor maximo: 1000 (itens 2 e 3)
 Entrada: 10 / 3 / 6 5 4 / 12 10 8
 Saída: Valor maximo: 18 (itens 2 e 3, peso 9)
 Entrada: 5 / 2 / 10 20 / 3 5
 Saída: Valor maximo: 0 (nada cabe)
 Entrada: 7 / 1 / 7 / 20
 Saída: Valor maximo: 20

Perguntas: quantos subconjuntos são testados para `n` itens? Por que só é viável para `n` pequeno?

4. Divisão e Conquista

1. **[Fácil]** Maior elemento (divisão e conquista). Escreva `int maior_dc(int vet[], int inicio, int fim)`: se o trecho tem 1 ou 2 elementos, devolva o maior entre eles; senão divida ao meio, resolva as duas metades recursivamente e devolva o maior dos dois resultados.

Entrada: `n`; os `n` elementos.

Saída: a linha Maior: X.

Casos de teste:

Entrada: 8 / 12 5 88 3 45 67 1 90 -> Saída: Maior: 90
 Entrada: 2 / 4 4 -> Saída: Maior: 4
 Entrada: 1 / 7 -> Saída: Maior: 7

2. **[Fácil]** Busca binária recursiva. Escreva `int busca_binaria_rec(int vet[], int inicio, int fim, int x)`: se `inicio > fim` devolva -1; calcule `meio = (inicio + fim)/2`; se `vet[meio] == x` devolva `meio`; se `vet[meio] > x` busque à esquerda; senão busque à direita. **Atenção:** o vetor está ordenado.

Entrada: n; os n elementos em ordem crescente; x.

Saída: o índice de x ou -1.

Casos de teste:

```
Entrada: 8 / 5 8 10 15 20 22 25 30 / 8    -> Saida: 1
Entrada: 8 / 5 8 10 15 20 22 25 30 / 17   -> Saida: -1
Entrada: 8 / 5 8 10 15 20 22 25 30 / 30   -> Saida: 7
Entrada: 5 / 2 4 6 8 10 / 2               -> Saida: 0
```

3. **[Fácil] Menor elemento (divisão e conquista).** Escreva `int menor_dc(int vet[], int inicio, int fim)` (o análogo do `maior_dc` para o menor elemento), com a mesma estratégia de divisão ao meio.

Entrada: n; os n elementos.

Saída: a linha Menor: X.

Casos de teste:

```
Entrada: 8 / 12 5 88 3 45 67 1 90    -> Saida: Menor: 1
Entrada: 3 / 9 2 5                   -> Saida: Menor: 2
Entrada: 1 / 42                      -> Saida: Menor: 42
```

4. **[Médio] Mergesort.** Implemente `void intercala(int a[], int p, int q, int r)` (fusione os trechos ordenados `[p..q]` e `[q+1..r]` em um vetor auxiliar e copie de volta) e `void mergesort(int a[], int p, int r)`: se `p < r`, divida ao meio em `q`, ordene as duas metades recursivamente e intercale. A main imprime o vetor após cada intercalação.

Entrada: n; os n elementos.

Saída: o vetor após cada intercalação; a última linha é o vetor ordenado.

Casos de teste:

```
Entrada: 4 / 8 3 7 1
Saida:   3 8 1 7
(intercala [3,8] e [1,7] das metades: após ordenar metades)
      1 3 7 8
(intercalação final)
```

5. **[Difícil] Quicksort.** Implemente `void quicksort(int a[], int e, int d)` com pivô `x = a[(e+d)/2]`: avance i enquanto `a[i] < x`, recue j enquanto `x < a[j]`, e se `i <= j` troque `a[i]` com `a[j]` (atualizando `i++`, `j--`); repita até `i > j`; recorra em `[e..j]` e `[i..d]`. A main imprime o vetor após cada partição completa.

Entrada: n; os n elementos.

Saída: o vetor após cada partição; a última linha é o vetor ordenado.

Casos de teste:

```
Entrada: 6 / 55 44 22 11 66 33
Saida:   33 11 22 44 66 55
(após a 1a particao: [22 11] | 44 | [66 55], após trocas parciais)
      11 22 33 44 55 66
(vetor ordenado)
```


Perguntas: (a) o quicksort tem etapa de combinação? (b) o que ocorre com o desempenho se o vetor já está ordenado e o pivô é sempre o elemento mediano? E se fosse o primeiro elemento?

5. Algoritmos Gulosos

1. **[Fácil] Problema do troco.** Escreva `void troco(int valor)` que, usando notas de 100, 50, 20, 10, 5, 2 e 1, imprime quantas notas de cada valor são usadas para compor `valor` com o menor número de notas. Formato: `<qtd> nota(s) de R$<valor>`. Imprima só as notas usadas (quantidade > 0). Pare quando o valor restante chegar a 0.

Entrada: um inteiro `valor` ≥ 0 .

Saída: uma linha por tipo de nota usada, da maior para a menor.

Casos de teste:

Entrada: 187 ->

Saída: 1 nota(s) de R\$100 / 1 nota(s) de R\$50 /

1 nota(s) de R\$20 / 1 nota(s) de R\$10 /

1 nota(s) de R\$5 / 1 nota(s) de R\$2

Entrada: 63 ->

Saída: 1 nota(s) de R\$50 / 1 nota(s) de R\$10 /

1 nota(s) de R\$2 / 1 nota(s) de R\$1

Entrada: 0 -> Saída: (nada)

2. **[Fácil] Máximo de arquivos no pendrive.** Escreva `int max_arquivos(int capacidade, int tamanhos[], int n)` que devolve o número máximo de arquivos que cabem no pendrive. Estratégia gulosa: ordene os tamanhos em ordem crescente e vá colocando arquivos enquanto couber.

Entrada: capacidade; n; os n tamanhos.

Saída: a linha Arquivos: X.

Casos de teste:

Entrada: 100 / 5 / 30 10 20 50 5

-> Saída: Arquivos: 4 (5, 10, 20, 30; sobram 35)

Entrada: 10 / 3 / 8 9 4

-> Saída: Arquivos: 1 (só o de 4 cabe com folga para nenhum outro)

Entrada: 50 / 4 / 10 10 10 10

-> Saída: Arquivos: 4

3. **[Fácil] Mochila fracionária (cálculo manual).** Dados $v = 60, 150, 120, 200$, $w = 10, 20, 30, 40$ e capacidade $W = 50$, resolva no papel pelas três estratégias: (a) maior valor primeiro; (b) menor peso primeiro; (c) maior razão valor/peso. Para cada uma indique as frações f_i escolhidas e o valor total. Não é preciso programar.

Saída: para cada estratégia: frações e valor total.

Casos de teste:

(a) maior valor primeiro:
 $f = \{0, 1/2, 0, 1\} \rightarrow \text{valor } 275$
 (b) menor peso primeiro:
 $f = \{1, 1, 2/3, 0\} \rightarrow \text{valor } 290$
 (c) maior razão v/w :
 $r = \{6, 7.5, 4, 5\}; f = \{1, 1, 0, 1/2\} \rightarrow$
 valor 310 (ÓTIMO)

Perguntas: qual estratégia é a correta? Por que a gulosa funciona na mochila fracionária mas não na booleana?

4. **[Médio] Escalonamento de intervalos (iterativo).** Usando `typedef struct` `int inicio; int fim; intervalo;`, escreva `void escalonamento(int n, intervalo evento[])`: ordene os eventos por fim crescente (desempate por início crescente), imprima o primeiro e depois, para cada evento seguinte, imprima-o somente se seu início for **estritamente maior** que o fim do último evento impresso.

Entrada: n ; n pares de inteiros `inicio fim`.

Saída: os intervalos escolhidos, um por linha, no formato `(inicio,fim)`.

Casos de teste:

Entrada: 11 / 1 4 / 3 5 / 0 6 / 5 7 /
 3 9 / 5 9 / 6 10 / 8 11 / 8 12 / 2 14 / 12 16
 Saída: (1,4) / (5,7) / (8,11) / (12,16)
 Entrada: 3 / 1 5 / 2 3 / 4 6
 Saída: (2,3) / (4,6)

5. **[Difícil] Reservas do laboratório.** Leia N ($1 \leq N \leq 20$) pedidos de reserva, cada um em uma linha no formato `HH:MM HH:MM` (início e término). Duas reservas são compatíveis se o início de uma for **maior ou igual** ao término da outra (pode começar exatamente quando a anterior termina). Estratégia gulosa: ordene por término crescente e escolha sempre a próxima compatível. Converta os horários para minutos desde 00:00 para comparar. Imprima apenas o número máximo de reservas.

Entrada: N ; N linhas com `HH:MM HH:MM`.

Saída: um inteiro: o número máximo de reservas atendidas.

Casos de teste:

Entrada: 5 / 13:00 17:00 / 9:40 11:20 / 11:00 12:40 / 14:00 16:00 /
 7:40 11:00 $\rightarrow$ Saída: 3
 Entrada: 2 / 9:00 10:00 / 10:00 11:00
 $\rightarrow$ Saída: 2 (encadeamento perfeito)
 Entrada: 3 / 8:00 12:00 / 9:00 10:00 / 10:00 11:00
 $\rightarrow$ Saída: 2

6. Programação Dinâmica

1. **[Fácil] Fibonacci (bottom-up).** Escreva `long long fib_bottomup(int n)` por tabulação: `tabela[0] = 0, tabela[1] = 1` e, para i de 2 até n , `tabela[i] = tabela[i-1] + tabela[i-2]`. Devolva `tabela[n]`.

Entrada: um inteiro $n \geq 0$.

Saída: a linha $\text{Fib}(n) = X$.

Casos de teste:

```
Entrada: 0    -> Saída: Fib(0) = 0
Entrada: 1    -> Saída: Fib(1) = 1
Entrada: 10   -> Saída: Fib(10) = 55
Entrada: 20   -> Saída: Fib(20) = 6765
Entrada: 50   -> Saída: Fib(50) = 12586269025
```

2. **[Fácil] Fibonacci (top-down).** Escreva `long long fib_topdown(int n)` por memoization com uma tabela global inicializada com um sentinela (ex.: -1): se $n \leq 1$ devolva n ; se `tabela[n]` já foi calculada, devolva-a; senão calcule recursivamente, **armazene** e devolva.

Entrada: um inteiro $n \geq 0$.

Saída: a linha $\text{Fib}(n) = X$.

Casos de teste:

```
Entrada: 0    -> Saída: Fib(0) = 0
Entrada: 10   -> Saída: Fib(10) = 55
Entrada: 30   -> Saída: Fib(30) = 832040
```

3. **[Fácil] Caracterização (teórico).** Cite e explique as duas características que um problema deve apresentar para admitir solução por programação dinâmica: *subestrutura ótima* e *sobreposição de subproblemas*. Ilustre ambas com a recorrência $F_n = F_{n-1} + F_{n-2}$.

Saída: resposta escrita com: (a) definição das duas características; (b) desenho da árvore de chamadas de `fib(4)` com os nós repetidos circutados.

Casos de teste:

```
fib(4) chama fib(3) e fib(2);
fib(3) chama fib(2) e fib(1);
fib(2) chama fib(1) e fib(0);
Subproblemas repetidos: fib(2) aparece 2x,
fib(1) aparece 3x, fib(0) aparece 2x.
```

4. **[Médio] Mochila booleana com PD.** Use uma tabela $M[i][c]$ = melhor valor usando os itens $1..i$ com capacidade c , preenchida *bottom-up*: $M[0][c] = 0$ para todo c ; para cada item i e capacidade c , se $p[i] > c$ então $M[i][c] = M[i-1][c]$, senão $M[i][c] = \max(M[i-1][c], v[i] + M[i-1][c - p[i]])$. Devolva $M[n][c]$.

Entrada: capacidade c ; n ; os n pesos; os n valores.

Saída: a linha Valor maximo: X .

Casos de teste:

Entrada: 50 / 5 / 40 30 20
 10 20 / 840 600 400 100 300 ->
 Saida: Valor maximo: 1000
 Entrada: 7 / 4 / 1 3 4 5 / 1 4 5 7
 -> Saida: Valor maximo: 9
 Entrada: 4 / 3 / 4 5 6 / 5 6 7
 -> Saida: Valor maximo: 7

Perguntas: qual a complexidade em função de n e c ? Compare com a força bruta (2^n).

5. **[Difícil] Multiplicação de cadeia de matrizes.** Para $A_1(4 \times 10)$, $A_2(10 \times 3)$, $A_3(3 \times 12)$, $A_4(12 \times 20)$ (vetor de dimensões $p = \{4, 10, 3, 12, 20\}$), preencha a tabela $M[i][j]$ *bottom-up* com $M[i][j] = \min_{i \leq k < j} \{M[i, k] + M[k + 1, j] + p_{i-1}p_kp_j\}$, anotando o k vencedor de cada célula. Preencha primeiro a diagonal ($M[i][i] = 0$), depois subsequências de tamanho 2, 3, 4.

Entrada: nenhuma (cálculo no papel; opcionalmente implemente).

Saída: a tabela M completa com os valores de k e a parentização ótima.

Casos de teste:

$M[1][2] = 120$ ($k=1$) $M[2][3] = 360$ ($k=2$) $M[3][4] = 720$ ($k=3$)
 $M[1][3] = 264$ ($k=2$) $M[2][4] = 1320$ ($k=3$)
 $M[1][4] = 1080$ ($k=2$) -> parentizacao otima: $(A_1 \times A_2) \times (A_3 \times A_4)$

2 Lista B

1. Funções

1. **[Fácil] Impressão de vetor.** Escreva `void imprime_vetor(int v[], int n)` que imprime o vetor no formato `[a, b, c]`: abre colchete, elementos separados por vírgula e espaço, fecha colchete, e quebra de linha ao final.

Entrada: `n`; os `n` elementos.

Saída: uma linha com o vetor no formato especificado.

Casos de teste:

Entrada: 4 / 3 7 1 9 -> Saída: [3, 7, 1, 9]

Entrada: 1 / 5 -> Saída: [5]

Entrada: 3 / -1 0 -2 -> Saída: [-1, 0, -2]

2. **[Fácil] Média de dois números.** Escreva `float media2(float a, float b)` que devolve a média aritmética. A `main` lê dois reais e imprime `Media: X` com exatamente 2 casas decimais (use `%.2f`).

Entrada: dois números reais.

Saída: a linha `Media: X` (2 casas decimais).

Casos de teste:

Entrada: 7.0 3.0 -> Saída: Media: 5.00

Entrada: 1.5 2.5 -> Saída: Media: 2.00

Entrada: -4 10 -> Saída: Media: 3.00

3. **[Fácil] Quadrado (passagem por valor).** Escreva `void quadrado(int x)` que faz `x = x * x` dentro da função e imprime `Quadrado: Y`. A `main` lê `n`, chama `quadrado(n)` e imprime `Original: n` após a chamada.

Entrada: um inteiro `n`.

Saída: duas linhas: `Quadrado: n2` e `Original: n`.

Casos de teste:

Entrada: 6 -> Saída: Quadrado: 36 / Original: 6

Entrada: -5 -> Saída: Quadrado: 25 / Original: -5

Perguntas: o valor original mudou? Explique o que acontece com a cópia temporária criada pela passagem por valor.

4. **[Médio] Conversão radianos para graus.** Escreva `void converte(float *angulo)` que recebe um ângulo em radianos **por referência** e substitui o conteúdo apontado pelo valor em graus (`graus = radianos × 180/π`). A `main` lê o valor em radianos, chama a função com o endereço da variável e imprime o resultado com 2 casas decimais.

Entrada: um número real (radianos).

Saída: a linha `Graus: X` (2 casas decimais).

Casos de teste:

Entrada: 3.14159 -> Saída: Graus: 180.00
Entrada: 1.5708 -> Saída: Graus: 90.00
Entrada: 0 -> Saída: Graus: 0.00

5. **[Difícil] Análise de vetor por referência.** Escreva `void analisa(int vet[], int n, int *pares, int *impares, int *soma)` que devolve por referência: quantos elementos são pares, quantos são ímpares e a soma de todos. A `main` lê `n` e os `n` elementos do teclado e imprime três linhas.

Entrada: `n`; os `n` inteiros.

Saída: Pares: `X`, Impares: `Y`, Soma: `Z` (uma por linha).

Casos de teste:

Entrada: 6 / 4 7 2 9 1 6 -> Saída: Pares: 3 / Impares: 3 / Soma: 29
Entrada: 4 / 2 4 6 8 -> Saída: Pares: 4 / Impares: 0 / Soma: 20
Entrada: 3 / 1 3 5 -> Saída: Pares: 0 / Impares: 3 / Soma: 9

2. Recursividade

1. **[Fácil] Fibonacci recursivo.** Escreva a função recursiva `long long fib(int n)` com solução trivial `fib(0) = 0`, `fib(1) = 1` e solução geral `fib(n) = fib(n-1) + fib(n-2)`.

Entrada: um inteiro `n` ≥ 0 .

Saída: a linha `Fib(n) = X`.

Casos de teste:

Entrada: 0 -> Saída: `Fib(0) = 0`
Entrada: 1 -> Saída: `Fib(1) = 1`
Entrada: 10 -> Saída: `Fib(10) = 55`
Entrada: 15 -> Saída: `Fib(15) = 610`

2. **[Fácil] Soma de vetor.** Escreva a função recursiva `int somavet(const int vet[], int n)`: solução trivial `n == 1` devolve `vet[0]`; solução geral `vet[n-1] + somavet(vet, n-1)`.

Entrada: `n`; os `n` inteiros.

Saída: a linha Soma: `X`.

Casos de teste:

Entrada: 4 / 4 2 7 1 -> Saída: Soma: 14
Entrada: 1 / 10 -> Saída: Soma: 10
Entrada: 5 / -1 1 -1 1 0 -> Saída: Soma: 0

3. **[Fácil] MDC (algoritmo de Euclides).** Escreva a função recursiva `int mdc(int a, int b)`: solução trivial `b == 0` devolve `a`; solução geral `mdc(b, a % b)`.

Entrada: dois inteiros positivos `a` e `b`.

Saída: a linha MDC: `X`.

Casos de teste:

Entrada: 48 36 -> Saida: MDC: 12
Entrada: 7 3 -> Saida: MDC: 1
Entrada: 100 10 -> Saida: MDC: 10
Entrada: 13 13 -> Saida: MDC: 13

4. **[Médio] Multiplicação por somas sucessivas.** Escreva a função recursiva `int mult(int a, int b)` que calcula $a \times b$ somando: solução trivial `b == 1` devolve `a`; solução geral `a + mult(a, b-1)`. Assuma `b >= 1`.

Entrada: dois inteiros `a` e `b >= 1`.

Saída: a linha Produto: X.

Casos de teste:

Entrada: 4 3 -> Saida: Produto: 12
Entrada: 5 1 -> Saida: Produto: 5
Entrada: 7 6 -> Saida: Produto: 42
Entrada: -3 4 -> Saida: Produto: -12

Perguntas: essa recursão é de cauda (a chamada recursiva é a última operação)?
Se sim, o que os slides recomendam nesse caso?

5. **[Difícil] Palíndromo recursivo.** Escreva a função recursiva `bool palindromo(const char *s, int i, int j)`: se `i >= j` devolva `true` (solução trivial); se `s[i] != s[j]` devolva `false`; senão devolva `palindromo(s, i+1, j-1)`. A `main` lê uma palavra e imprime `Palindromo` ou `Nao palindromo` (chame com `palindromo(s, 0, strlen(s)-1)`).

Entrada: uma palavra (sem espaços).

Saída: `Palindromo` ou `Nao palindromo`.

Casos de teste:

Entrada: arara -> Saida: Palindromo
Entrada: banana -> Saida: Nao palindromo
Entrada: a -> Saida: Palindromo
Entrada: 12321 -> Saida: Palindromo
Entrada: (vazio) -> Saida: Palindromo

3. Força Bruta

1. **[Fácil] Maior elemento (força bruta).** Escreva `int maior_fb(int n, int vet[])` que percorre o vetor uma única vez mantendo o maior valor visto. Comece assumindo que o maior é `vet[0]` e percorra de 1 até `n-1`.

Entrada: `n`; os `n` elementos.

Saída: a linha Maior: X.

Casos de teste:

Entrada: 6 / 12 5 88 3 45 67 -> Saida: Maior: 88
Entrada: 2 / 9 9 -> Saida: Maior: 9
Entrada: 5 / -5 -2 -9 -1 -7 -> Saida: Maior: -1

2. **[Fácil] Bubble sort.** Escreva `void bubblesort(int n, int a[])`: faça $n-1$ passagens; em cada passagem, para j de 0 até $n-2-i$, troque $a[j]$ com $a[j+1]$ se estiverem fora de ordem. A main imprime o vetor após cada passagem.

Entrada: n ; os n elementos.

Saída: o vetor após cada uma das $n-1$ passagens.

Casos de teste:

Entrada: 5 / 5 1 4 2 8

Saída: 1 4 2 5 8 (passagem 0)

1 2 4 5 8 (passagem 1)

1 2 4 5 8 (passagem 2, sem trocas)

1 2 4 5 8 (passagem 3)

3. **[Fácil] Insertion sort.** Escreva `void insercao(int n, int a[])`: para cada i de 1 até $n-1$, guarde $aux = a[i]$ e desloque os elementos maiores que aux uma posição à direita, inserindo aux na posição correta. Imprima o vetor após cada inserção.

Entrada: n ; os n elementos.

Saída: o vetor após cada passo.

Casos de teste:

Entrada: 4 / 44 55 12 42

Saída: 44 55 12 42 (passo 0: 55 já estava ordenado)

12 44 55 42 (passo 1: 12 inserido no início)

12 42 44 55 (passo 2)

4. **[Médio] Contagem de substrings.** Escreva `int conta_substring(const char *s1, const char *s2)` que conta quantas vezes $s2$ ocorre em $s1$, considerando apenas ocorrências **não sobrepostas**: após encontrar uma ocorrência em i , a próxima busca começa em $i + m$.

Entrada: duas strings $s1$ e $s2$.

Saída: a linha `Ocorrencias: X`.

Casos de teste:

Entrada: aaaa aa -> Saída: Ocorrencias: 2

Entrada: unisantos nt -> Saída: Ocorrencias: 1

Entrada: unisantos s -> Saída: Ocorrencias: 2

Entrada: abc abcabc abc -> Saída: Ocorrencias: 2

Entrada: abc xyz -> Saída: Ocorrencias: 0

5. **[Difícil] Pontos mais próximos (força bruta).** Usando `struct ponto { int x; int y; ;}`, escreva `double pontos_mais_proximos(int n, struct ponto p[])` que testa **todo par** (i, j) com $i < j$, calcula a Euclidiana $d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ e devolve a menor delas. A main lê n e os n pontos e imprime a distância com 2 casas decimais.

Entrada: n ; n linhas com x y .

Saída: a linha `Distancia: X (2 casas decimais)`.

Casos de teste:

Entrada: 4 / 1 2 / 4 6 / 1 3 / 8 8
 -> Saída: Distancia: 1.00 (par (1,2)-(1,3))
 Entrada: 2 / 0 0 / 3 4
 -> Saída: Distancia: 5.00
 Entrada: 3 / 0 0 / 10 0 / 0 10
 -> Saída: Distancia: 10.00

Perguntas: quantos pares são testados para n pontos? Por que fica impraticável para n grande?

4. Divisão e Conquista

1. **[Fácil] Menor elemento (divisão e conquista).** Escreva `int min_dc(int vet[], int inicio, int fim)`: para trechos de 1 ou 2 elementos devolva o menor diretamente; senão divida ao meio e devolva o menor entre as duas metades resolvidas recursivamente.

Entrada: n ; os n elementos.

Saída: a linha Menor: X.

Casos de teste:

Entrada: 8 / 12 5 88 3 45 67 1 90 -> Saída: Menor: 1
 Entrada: 4 / 9 2 5 7 -> Saída: Menor: 2
 Entrada: 2 / 3 3 -> Saída: Menor: 3

2. **[Fácil] Simulação da busca binária.** Suponha que $v[i] = i$ para todo i (vetor ordenado contendo os próprios índices). Execute a busca binária **no papel** para os três casos ao lado. Para cada caso, escreva uma tabela com as colunas **inicio**, **final** e **meio** a cada iteração, e indique o que é devolvido.

Entrada: parâmetros n e x conforme cada caso.

Saída: tabela de iterações e valor devolvido.

Casos de teste:

(a) $n = 9, x = 3$ -> encontra o índice 3
 (b) $n = 14, x = 7$ -> encontra o índice 7
 (c) $n = 15, x = 7$ -> encontra o índice 7
 (desafio) em (a), (b) e (c),
 quantas iterações foram feitas? O que muda entre n par e n ímpar?

3. **[Fácil] Busca binária com contagem de divisões.** Escreva `int busca_bin_conta(int vet[], int n, int x, int *divisoes)` (versão iterativa) que devolve o índice de x (ou -1) e, por referência, quantas vezes o intervalo foi dividido ao meio.

Entrada: n ; os n elementos ordenados; x .

Saída: o índice e o número de divisões.

Casos de teste:

Entrada: 8 / 5 8 10 15 20 22 25 30 / 25 -> Saída: índice 6, 2 divisões
 Entrada: 8 / 5 8 10 15 20 22 25 30 / 17 -> Saída: índice -1, 3 divisões
 Entrada: 1 / 42 / 42 -> Saída: índice 0, 0 divisões

4. **[Médio] Simulação do mergesort.** Para o vetor $v = 55, 44, 22, 11, 66, 33$, desenhe no papel: (1) a árvore de divisões até chegar a vetores de 1 elemento; (2) o resultado de cada intercalação, do nível mais baixo até o vetor completo. Depois implemente `mergesort + intercala` e confira se o papel bate com a execução.

Entrada: $n = 6$; os 6 elementos.

Saída: desenho da árvore + vetor após cada intercalação; final 11 22 33 44 55 66.

Casos de teste:

```
Divisoes: [55 44 22] [11 66 33] -> [55 44] [22] | [11 66] [33] -> ...
Intercalacoes de 2: [44 55] [22] | [11 66] [33]
Intercalacoes de 3: [22 44 55] [11 33 66]
Intercalacao final: 11 22 33 44 55 66
```

5. **[Difícil] Simulação do quicksort.** Simule no papel o quicksort com pivô no elemento mediano para $v = 44, 55, 12, 42, 94, 18, 67$. Para cada chamada, indique: o subvetor, o pivô x , a posição final de i e j e o vetor resultante. Depois implemente e confira.

Entrada: $n = 7$; os 7 elementos.

Saída: estado do vetor após cada partição, com identificação das chamadas recursivas.

Casos de teste:

```
1a chamada (0..6): pivo = 42; apos particoes e trocas:
18 12 | 42 | 55 94 67 -> recursao em [0..1] e [3..6]
Subvetor esq [18 12]: pivo = media(18,12); troca -> 12 18
Subvetor dir [55 94 67]: pivo = 94; apos trocas:
55 67 94 -> recursao em [3..4] e [5..6]
Vetor final: 12 18 42 44 55 67 94
```

Perguntas: quantas partições e quantas trocas ocorreram no total?

5. Algoritmos Gulosos

1. **[Fácil] Troco com moedas.** Escreva `void troco_moedas(int centavos)` usando moedas de 50, 25, 10, 5 e 1 centavos, imprimindo `<qtd> moeda(s) de <valor>c` para cada moeda usada, da maior para a menor.

Entrada: um inteiro `centavos` ≥ 0 .

Saída: uma linha por tipo de moeda usada.

Casos de teste:

```
Entrada: 91 -> Saida: 1 moeda(s) de 50c / 1 moeda(s) de 25c /
1 moeda(s) de 10c / 1 moeda(s) de 5c / 1 moeda(s) de 1c
Entrada: 30 -> Saida: 1 moeda(s) de 25c /
1 moeda(s) de 5c
Entrada: 4 -> Saida: 4 moeda(s) de 1c
```

2. **[Fácil]** **Verificação gulosa de pagamento.** Escreva `bool pode_pagar(int valor, int notas[], int n)` que verifica se é possível compor valor **exatamente** com as notas disponíveis: a cada passo, use o máximo possível da maior nota e vá descendo; se sobrar valor não composto, devolva `false`. A `main` imprime `Possivel` ou `Impossivel`.

Entrada: valor; n; as n notas em ordem decrescente.

Saída: `Possivel` ou `Impossivel`.

Casos de teste:

Entrada: 87 / 5 / 50 20 10 5 2 -> Saída: `Possivel` (50+20+10+5+2)

Entrada: 87 / 3 / 50 20 20 -> Saída: `Impossivel` (sobram 17)

Entrada: 0 / 2 / 10 5 -> Saída: `Possivel` (valor zero)

3. **[Fácil]** **Mochila fracionária (cálculo manual).** Dados $v = 100, 200, 120$, $w = 10, 20, 30$ e $W = 50$, calcule no papel o valor máximo pela estratégia da maior razão valor/peso. Calcule $r_i = v_i/w_i$, ordene, e preencha a mochila inteiramente até sobrar espaço; nesse caso pegue a fração do próximo item.

Saída: razões ordenadas, frações f_i de cada item e valor total.

Casos de teste:

$r = \{10, 10, 4\}$ -> ordem: itens 1 e 2 (empate, qualquer), depois item 3

$f = \{1, 1, 2/3\}$ -> peso: $10 + 20 + (2/3)*30 = 50$ (capacidade exata)

valor = $100 + 200 + (2/3)*120 = 380$ (ÓTIMO)

4. **[Médio]** **Escalonamento de intervalos (recursivo).** Implemente a versão recursiva: coloque um intervalo fictício 0, 0 na posição 0 do vetor, ordene os eventos reais (posições 1 a n) por fim crescente, e escreva `void escalonamento_rec(intervalo evento[], int e, int d)` que encontra o primeiro evento após e com início maior que `evento[e].fim`, imprime-o e recorre a partir dele.

Entrada: n; n pares início fim.

Saída: os intervalos escolhidos, um por linha, no formato (início,fim).

Casos de teste:

Entrada: 3 / 1 5 / 2 3 / 4 6 -> Saída: (2,3) / (4,6)

Entrada: 4 / 1 2 / 2 3 / 3 4 / 0 10 -> Saída: (1,2) / (3,4)

Entrada: 2 / 0 100 / 1 2 -> Saída: (1,2)

5. **[Difícil]** **Reservas do laboratório (com impressão dos horários).** Mesmas regras do problema da Lista A (reservas HH:MM HH:MM, compatíveis se início $\geq$ término), mas a saída deve conter **primeiro o número total e depois os horários escolhidos**, um por linha, no formato original.

Entrada: N; N linhas HH:MM HH:MM.

Saída: um inteiro, seguido dos horários escolhidos.

Casos de teste:

Entrada: 6 / 9:00 10:00 / 10:00 11:30 /
 9:30 12:00 / 13:00 14:00 / 14:30 15:00 /
 8:00 9:30
 Saída: 4
 8:00 9:30
 10:00 11:30
 13:00 14:00
 14:30 15:00

6. Programação Dinâmica

1. **[Fácil] Fibonacci com impressão da sequência.** Escreva void `fib_sequencia(int n)` por tabulação que imprime todos os termos de F_0 a F_n na mesma linha, separados por espaço.

Entrada: um inteiro $n \geq 0$.

Saída: os $n+1$ termos separados por espaço.

Casos de teste:

Entrada: 0 -> Saída: 0
 Entrada: 1 -> Saída: 0 1
 Entrada: 10 -> Saída: 0 1 1 2 3 5 8 13 21 34 55

2. **[Fácil] Fibonacci (top-down).** Escreva long long `fib_topdown(int n)` por memoization: tabela global inicializada com -1; se $n \leq 1$ devolva n ; se `tabela[n] != -1` devolva `tabela[n]`; senão calcule, armazene e devolva.

Entrada: um inteiro $n \geq 0$.

Saída: a linha `Fib(n) = X`.

Casos de teste:

Entrada: 0 -> Saída: Fib(0) = 0
 Entrada: 2 -> Saída: Fib(2) = 1
 Entrada: 10 -> Saída: Fib(10) = 55
 Entrada: 30 -> Saída: Fib(30) = 832040

3. **[Fácil] Top-down vs bottom-up (teórico).** Complete a tabela comparativa entre as abordagens *top-down* (memoization) e *bottom-up* (tabulação) nos quesitos: facilidade de pensar o problema, simplicidade do código, quais subproblemas são resolvidos e o que é armazenado na tabela.

Saída: tabela preenchida com 4 linhas (uma por quesito) e 2 colunas (uma por abordagem).

Casos de teste:

Exemplo de linha: 'Subproblemas resolvidos' |
 top-down: apenas os necessarios |
 bottom-up: todos, do menor ao maior

4. **[Médio] Mochila booleana com PD.** Preencha a tabela $M[i][c]$ *bottom-up* para $p = 1, 3, 4, 5$, $v = 1, 4, 5, 7$ e capacidade $c = 7$, com a regra: se $p[i] > c$, $M[i][c] = M[i-1][c]$; senão $M[i][c] = \max(M[i-1][c], v[i] + M[i-1][c - p[i]])$. Apresente a tabela completa (5 linhas $\times$ 8 colunas, de $c = 0$ a 7) e o valor ótimo.

Saída: a tabela completa e o valor ótimo.

Casos de teste:

$M[1][c] = c$ (item 1: peso 1, valor 1 $\rightarrow$ cabe em toda capacidade)

$M[4][7] = 9 \rightarrow$ itens de peso 3 (valor 4) e 4 (valor 5)

Otimo: 9

5. **[Difícil] Multiplicação de cadeia de matrizes.** Para $A_1(10 \times 30)$, $A_2(30 \times 5)$, $A_3(5 \times 60)$ ($p = \{10, 30, 5, 60\}$), preencha a tabela $M[i][j]$ *bottom-up* com $M[i][j] = \min_{i \leq k < j} \{M[i][k] + M[k+1][j] + p_{i-1}p_kp_j\}$. Mostre o cálculo completo de cada célula fora da diagonal.

Saída: tabela 3×3 , custo mínimo e parentização ótima.

Casos de teste:

$M[1][2] = 10 \times 30 \times 5 = 1500$

$M[2][3] = 30 \times 5 \times 60 = 9000$

$M[1][3]: k=1 \rightarrow 0 + 9000 + 10 \times 30 \times 60 = 27000;$

$k=2 \rightarrow 1500 + 0 + 10 \times 5 \times 60 = 4500$ ($k=2$ vence)

Custo mínimo: 4500 $\rightarrow$ parentizacao: $(A_1 \times A_2) \times A_3$

3 Lista C

1. Funções

1. **[Fácil] Cabeçalho.** Escreva `void cabecalho(const char *titulo)` que imprime o título entre duas linhas de = (ex.: ===== CALCULADOR =====, com `strlen(titulo)` + 4 caracteres = em cada linha). A `main` lê uma frase de até 50 caracteres (sem espaços) e chama a função.

Entrada: uma palavra ou frase sem espaços (até 50 caracteres).

Saída: linha de =, título entre espaços, linha de =.

Casos de teste:

Entrada: PROVA ->

Saída: =====

PROVA

=====

Entrada: C ->

Saída: =====

C

=====

2. **[Fácil] Valor absoluto.** Escreva `int absoluto(int x)` que devolve $|x|$ **sem usar** nenhuma função da `math.h` (use uma condição).

Entrada: um inteiro.

Saída: a linha Absoluto: X.

Casos de teste:

Entrada: -7 -> Saída: Absoluto: 7

Entrada: 3 -> Saída: Absoluto: 3

Entrada: 0 -> Saída: Absoluto: 0

3. **[Fácil] Incremento (passagem por valor).** Escreva `void incrementa(int x)` que faz `x++` dentro da função. A `main` declara `n = 5`, chama `incrementa(n)` e imprime `n`.

Entrada: nenhuma (`n = 5` fixo na `main`).

Saída: o valor impresso de `n`.

Casos de teste:

Saída: 5

Perguntas: por que não foi impresso 6? O que o parâmetro formal `x` representa na memória?

4. **[Médio] Divisão inteira por referência.** Escreva `void divisao(int a, int b, int *q, int *r)` que devolve por referência o quociente e o resto de `a` dividido por `b`, **sem usar** os operadores `/` e `%`: subtraia `b` de `a` repetidamente, contando quantas subtrações foram feitas; o que sobrar é o resto. Assuma `a >= 0` e `b > 0`.

Entrada: dois inteiros $a \geq 0$ e $b > 0$.

Saída: Quociente: X e Resto: Y.

Casos de teste:

Entrada: 17 5 -> Saída: Quociente: 3 / Resto: 2

Entrada: 20 4 -> Saída: Quociente: 5 / Resto: 0

Entrada: 3 10 -> Saída: Quociente: 0 / Resto: 3

5. **[Difícil] Soma de matrizes.** Escreva `void soma_matrizes(int a[][10], int b[][10], int c[][10], int n)` que soma duas matrizes $n \times n$ (limite máximo 10). A main lê n e os elementos de a e b , chama a função e imprime a matriz c com uma linha por linha da matriz e elementos separados por espaço.

Entrada: n ($1 \leq n \leq 10$); os n^2 elementos de a ; os n^2 elementos de b (linha a linha).

Saída: as n linhas da matriz soma.

Casos de teste:

Entrada: 2 / 1 2 / 3 4 / 5 6 / 7 8 -> Saída: 6 8 / 10 12

Entrada: 1 / 5 / 5 -> Saída: 10

2. Recursividade

1. **[Fácil] Fatorial.** Escreva a função recursiva `long long fat(int n)` e a main que lê n do teclado.

Entrada: um inteiro $n \geq 0$.

Saída: a linha Fatorial: X.

Casos de teste:

Entrada: 0 -> Saída: Fatorial: 1

Entrada: 6 -> Saída: Fatorial: 720

Entrada: 3 -> Saída: Fatorial: 6

2. **[Fácil] Potenciação.** Escreva a função recursiva `int pot(int x, int y)` com solução trivial $x^0 = 1$ e solução geral $x^y = x \cdot x^{y-1}$.

Entrada: dois inteiros $x, y \geq 0$.

Saída: a linha Resultado: X.

Casos de teste:

Entrada: 3 4 -> Saída: Resultado: 81

Entrada: 2 8 -> Saída: Resultado: 256

Entrada: 9 0 -> Saída: Resultado: 1

3. **[Fácil] Soma de vetor.** Escreva a função recursiva `int somavet(const int vet[], int n)` (solução trivial $n == 1$; solução geral $vet[n-1] + somavet(vet, n-1)$).

Entrada: n ; os n inteiros.

Saída: a linha Soma: X.

Casos de teste:

Entrada: 4 / 10 20 30 40 -> Saida: Soma: 100
Entrada: 2 / -3 3 -> Saida: Soma: 0

4. **[Médio] Par ou ímpar (recursão indireta).** Escreva `bool par(int n)` e `bool impar(int n)` com recursão indireta: `par` devolve `true` se `n == 0`, `false` se `n == 1`, senão devolve `impar(n-1)`; `impar` devolve `true` se `n == 1`, `false` se `n == 0`, senão devolve `par(n-1)`. A main lê um número e imprime `par` ou `impar`.

Entrada: um inteiro `n >= 0`.

Saída: a palavra `par` ou `impar`.

Casos de teste:

Entrada: 10 -> Saida: par
Entrada: 7 -> Saida: impar
Entrada: 0 -> Saida: par
Entrada: 1 -> Saida: impar

5. **[Difícil] Contagem de chamadas do Fibonacci.** Escreva o `fib` recursivo ingênuo com uma variável global `long long chamadas` incrementada no início de cada chamada. A main imprime, para cada `n`, o valor de `fib(n)` e o número de chamadas.

Entrada: um inteiro `n`.

Saída: duas linhas: `Fib(n) = X` e `Chamadas: Y`.

Casos de teste:

Entrada: 10 -> Saida: Fib(10) = 55 / Chamadas: 177
Entrada: 15 -> Saida: Fib(15) = 610 / Chamadas: 1973
Entrada: 20 -> Saida: Fib(20) = 6765 / Chamadas: 21891

Perguntas: (a) como o número de chamadas cresce? (b) qual subproblema é recalculado mais vezes? (c) o que a programação dinâmica faz para resolver isso?

3. Força Bruta

1. **[Fácil] Busca de caractere em string.** Escreva `int busca_char(const char *s, char c)` que retorna o índice da primeira ocorrência do caractere `c` em `s`, ou `-1`. Não use `strchr`.

Entrada: uma string e um caractere (separados por espaço).

Saída: o índice ou `-1`.

Casos de teste:

Entrada: unisantos s -> Saida: 2
Entrada: unisantos x -> Saida: -1
Entrada: aaa a -> Saida: 0

2. **[Fácil] Bubble sort.** Escreva `void bubblesort(int n, int a[])` e mostre o vetor após cada passagem para o caso ao lado.

Entrada: n ; os n elementos.

Saída: o vetor após cada passagem.

Casos de teste:

```
Entrada: 5 / 9 4 7 1 3
Saída:   4 7 1 3 9    (passagem 0)
         4 1 3 7 9    (passagem 1)
         1 3 4 7 9    (passagem 2)
         1 3 4 7 9    (passagem 3)
```

3. **[Fácil] Seleção direta com rastreamento.** Escreva `void selecao_direta(int n, int a[])` que, a cada passo i , encontra o índice k do menor elemento em $[i..n-1]$ e troca com $a[i]$. Imprima Passo i : $k = \langle \text{índice} \rangle$ seguido do vetor.

Entrada: n ; os n elementos.

Saída: $n-1$ linhas de rastreamento com o vetor.

Casos de teste:

```
Entrada: 5 / 9 4 7 1 3
Saída:   Passo 0: k = 3 -> 1 4 7 9 3
         Passo 1: k = 3 -> 1 3 7 9 4
         Passo 2: k = 4 -> 1 3 4 9 7
         Passo 3: k = 4 -> 1 3 4 7 9
```

4. **[Médio] String match com posição.** Escreva `int string_match_pos(const char *s1, const char *s2)` que retorna a **posição** (índice) da primeira ocorrência de $s2$ em $s1$, ou -1 (força bruta: tente cada alinhamento i de 0 até $n - m$).

Entrada: duas strings $s1$ e $s2$.

Saída: a posição ou -1 .

Casos de teste:

```
Entrada: unisantos san    -> Saída: 2
Entrada: unisantos tnt    -> Saída: -1
Entrada: unisantos uni    -> Saída: 0
Entrada: abc abc          -> Saída: 0
Entrada: abc abcd         -> Saída: -1    (m > n)
```

5. **[Difícil] Mochila booleana (iterativa, força bruta com bits).** Escreva `int mochila_iter(int p[], int v[], int c, int n)` que enumera todos os 2^n subconjuntos: para cada máscara i de 0 até $2^n - 1$, o bit j de i (testado com $(i \gg j) \% 2$) indica se o item j está no subconjunto; some peso e valor dos itens escolhidos e guarde o maior valor cujo peso não exceda c .

Entrada: capacidade c ; n ; os n pesos; os n valores.

Saída: a linha Valor maximo: X .

Casos de teste:

Entrada: 50 / 5 / 40 30 20 10 20 / 840 600 400 100 300
 -> Saída: Valor maximo: 1000
 Entrada: 7 / 4 / 1 3 4 5 / 1 4 5 7
 -> Saída: Valor maximo: 9
 Entrada: 3 / 2 / 4 5 / 10 20
 -> Saída: Valor maximo: 0

4. Divisão e Conquista

1. **[Fácil] Maior elemento (divisão e conquista).** Escreva `int maior_dc(int vet[], int inicio, int fim)`. Teste com o caso ao lado.

Entrada: n; os n elementos.

Saída: a linha Maior: X.

Casos de teste:

Entrada: 5 / 5 12 3 20 8
 -> Saída: Maior: 20
 Entrada: 2 / 7 7
 -> Saída: Maior: 7

2. **[Fácil] Busca binária iterativa.** Escreva `int busca_binaria_iter(int vet[], int n, int x)`: enquanto `inicio <= fim`, calcule `meio`; se `vet[meio] == x` devolva `meio`; se `vet[meio] > x` busque à esquerda (`final = meio - 1`); senão à direita (`inicio = meio + 1`).

Entrada: n; os n elementos ordenados; x.

Saída: o índice ou -1.

Casos de teste:

Entrada: 6 / 2 4 6 8 10 12 / 8 -> Saída: 3
 Entrada: 6 / 2 4 6 8 10 12 / 7 -> Saída: -1
 Entrada: 6 / 2 4 6 8 10 12 / 2 -> Saída: 0
 Entrada: 6 / 2 4 6 8 10 12 / 12 -> Saída: 5

3. **[Fácil] Menor elemento (divisão e conquista).** Escreva `int menor_dc(int vet[], int inicio, int fim)` com a mesma estratégia do `maior_dc`.

Entrada: n; os n elementos.

Saída: a linha Menor: X.

Casos de teste:

Entrada: 5 / 5 12 3 20 8 -> Saída: Menor: 3
 Entrada: 3 / 0 -1 9 -> Saída: Menor: -1

4. **[Médio] Mergesort completo.** Implemente mergesort e intercala e ordene o vetor ao lado. **Pergunta:** quantas intercalações ocorrem para um vetor de 5 elementos? E para n elementos (em função de n)?

Entrada: n; os n elementos.

Saída: o vetor ordenado.

Casos de teste:

Entrada: 5 / 9 7 5 3 1 -> Saída: 1 3 5 7 9
Entrada: 1 / 42 -> Saída: 42

5. **[Difícil] Quicksort com contadores.** Implemente o quicksort acrescentando duas variáveis globais `int trocas` e `int particoes`, incrementadas nos pontos correspondentes. Rode para `v1 = 55, 44, 22, 11, 66, 33` e para `v2 = 11, 22, 33, 44, 55, 66` (já ordenado) e compare os contadores.

Entrada: `n`; os `n` elementos.

Saída: vetor ordenado, Trocas: `X`, Particoes: `Y`.

Casos de teste:

Entrada: 6 / 55 44 22 11 66 33
-> Saída: 11 22 33 44 55 66 / Trocas e particoes conforme execucao
Entrada: 6 / 11 22 33 44 55 66
-> Saída: 11 22 33 44 55 66 / Trocas e particoes conforme execucao

Perguntas: o experimento revela algo sobre o pior caso do quicksort? O que muda se o vetor já está ordenado?

5. Algoritmos Gulosos

1. **[Fácil] Problema do troco.** Escreva `void troco(int valor)` com as notas 100, 50, 20, 10, 5, 2, 1. Teste com os casos ao lado.

Entrada: um inteiro `valor` ≥ 0 .

Saída: uma linha por nota usada.

Casos de teste:

Entrada: 63 -> Saída: 1 nota(s) de R\$50 /
1 nota(s) de R\$10 / 1 nota(s) de R\$2 / 1 nota(s) de R\$1
Entrada: 100 -> Saída: 1 nota(s) de R\$100
Entrada: 8 -> Saída: 1 nota(s) de R\$5 /
1 nota(s) de R\$2 / 1 nota(s) de R\$1

2. **[Fácil] Quando o guloso falha (contraexemplo).** (a) À mão: com notas de 5, 4 e 1, execute a estratégia gulosa para o valor 8: quantas notas são usadas e quais? (b) Qual é a solução ótima? (c) Escreva `void troco_geral(int valor, int notas[], int n)` que executa o guloso com as notas do vetor (em ordem decrescente) e imprime as notas usadas, confirmando o resultado de (a).

Entrada: para o código: `valor`; `n`; as `n` notas em ordem decrescente.

Saída: respostas de (a) e (b) + saída do código em (c).

Casos de teste:

(a) guloso: $5 + 1 + 1 + 1 = 8$ -> 4 notas
(b) ótimo: $4 + 4 = 8$ -> 2 notas
(c) Entrada: 8 / 3 / 5 4 1 ->
Saída: 1 nota(s) de 5 / 3 nota(s) de 1

Perguntas: por que o algoritmo guloso falha aqui? Qual propriedade do conjunto de notas 100,50,20,10,5,2,1 garante que o guloso funciona no sistema real?

3. **[Fácil] Mochila fracionária (cálculo manual).** Dados $v = 10, 40, 30, 50$, $w = 5, 4, 6, 3$ e $W = 10$, calcule o valor máximo pela maior razão valor/peso, indicando as frações f_i .

Saída: razões, ordem de escolha, frações e valor total.

Casos de teste:

```
r = {2, 10, 5, 16.67} -> ordem: item 4, item 2, item 3, item 1
f4 = 1
(peso 3), f2 = 1 (peso 4), f3 = 3/6 = 1/2
(peso 3) -> peso total 10
valor = 50 + 40 + (1/2)*30 = 105 (OTIMO)
```

4. **[Médio] Escalonamento de intervalos (programa completo).** Escreva o programa completo: leia n e n intervalos `inicio fim`, ordene com `qsort` usando `int compara(const void *a, const void *b)` (`fim` crescente, desempate por `inicio`) e aplique o guloso, imprimindo o número de eventos atendidos seguido dos intervalos.

Entrada: n ; n pares `inicio fim`.

Saída: o número de eventos e os intervalos escolhidos.

Casos de teste:

```
Entrada: 5 / 1 3 / 2 5 / 4 7 / 6 9 / 8 10
Saída:   3
        (1,3)
        (4,7)
        (8,10)
```

5. **[Difícil] Reservas do laboratório.** Leia N reservas `HH:MM HH:MM` e imprima apenas o número máximo atendível (`inicio  $\geq$  término da anterior`). Use a estratégia gulosa ordenando por término.

Entrada: N ; N linhas `HH:MM HH:MM`.

Saída: um inteiro.

Casos de teste:

```
Entrada: 4 / 8:00 9:00 / 9:00 10:30 / 9:15 9:45 / 10:00 11:00
-> Saída: 2
Entrada: 3 / 9:00 9:30 / 9:30 10:00 / 10:00 10:30
-> Saída: 3
Entrada: 2 / 10:00 12:00 / 11:00 11:30
-> Saída: 1
```

Perguntas: no primeiro caso, por que a reserva das 10:00 não pode ser atendida junto com as duas escolhidas?

6. Programação Dinâmica

1. **[Fácil] Fibonacci (bottom-up).** Escreva `long long fib_bottomup(int n)` por tabulação.

Entrada: um inteiro $n \geq 0$.

Saída: a linha $\text{Fib}(n) = X$.

Casos de teste:

Entrada: 0 -> Saída: $\text{Fib}(0) = 0$

Entrada: 15 -> Saída: $\text{Fib}(15) = 610$

Entrada: 25 -> Saída: $\text{Fib}(25) = 75025$

2. **[Fácil] Fibonacci (top-down).** Escreva `long long fib_topdown(int n)` por memoization com tabela global inicializada com -1.

Entrada: um inteiro $n \geq 0$.

Saída: a linha $\text{Fib}(n) = X$.

Casos de teste:

Entrada: 0 -> Saída: $\text{Fib}(0) = 0$

Entrada: 15 -> Saída: $\text{Fib}(15) = 610$

Entrada: 40 -> Saída: $\text{Fib}(40) = 102334155$

3. **[Fácil] Sobreposição de subproblemas (teórico).** Desenhe a árvore de chamadas de $\text{fib}(5)$ e responda: (a) quantas vezes cada subproblema $\text{fib}(0) \dots \text{fib}(3)$ é resolvido? (b) qual o número total de chamadas? (c) quantas chamadas sobriam com memoization?

Saída: árvore desenhada + três respostas numéricas.

Casos de teste:

$\text{fib}(3)$ x2, $\text{fib}(2)$ x3, $\text{fib}(1)$ x5, $\text{fib}(0)$ x3

Total de chamadas (incluindo a inicial): 15

Com memoization: apenas 5 chamadas de calculo ($\text{fib}(0)$ a $\text{fib}(4)$)

4. **[Médio] Mochila booleana com PD (com reconstrução).** Além do valor ótimo, imprima **quais itens foram escolhidos**: percorra a tabela M de (n, c) para trás — se $M[i][c] \neq M[i-1][c]$, o item i foi escolhido; passe para $(i-1, c - p[i])$; senão apenas decmente i . Dados: $p = 40, 30, 20, 10, 20$, $v = 840, 600, 400, 100, 300$, $c = 50$.

Entrada: capacidade c ; n ; os n pesos; os n valores.

Saída: Valor maximo: X e Itens: ... (índices, em ordem crescente).

Casos de teste:

Entrada: 50 / 5 / 40 30 20 10 20 / 840 600 400 100 300

Saída: Valor maximo: 1000

Itens: 2 3

5. **[Difícil] Multiplicação de cadeia de matrizes (6 matrizes).** Para $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$, $A_6(20 \times 25)$ ($p = \{30, 35, 15, 5, 10, 20, 25\}$), preencha a tabela $M[i][j]$ (6×6) *bottom-up*. Indique o k vencedor de $M[1, 6]$ e recupere a parentização ótima completa percorrendo as escolhas de k .

Saída: tabela completa, custo mínimo e parentização.

Casos de teste:

```
M[1][2] = 15750    M[2][3] = 2625    M[3][4] = 750
M[4][5] = 1000     M[5][6] = 5000
M[1][6] = 15125    (k = 3)
-> ((A1 x A2) x ((A3 x A4) x A5)) x
A6 (recupere percorrendo as escolhas de k a partir de M[1,6])
```

Referências

- [1] WIRTH, N. *Algoritmos e Estruturas de Dados*. LTC, 1999.
- [2] CORMEN, T. H. et al. *Algoritmos: teoria e prática*. Rio de Janeiro: Editora Campus, 2002.